

An Asymmetric Cascaded Framework for High-Throughput Network Intrusion Detection: Decoupling Deep Sequence Triage and Post-Hoc Interpretability

Piyush M. Borkar
Dept. of Artificial Intelligence
& Data Science
Marathwada Mitra Mandal's
Institute of Technology
Pune, India
piyushborkar.official@gmail.com

Varun Gada
Dept. of Computer Engineering
Marathwada Mitra Mandal's
Institute of Technology
Pune, India
varungada2004@gmail.com

Dr. Boppuru Rudra Prathap
Dept. of Computer Science
& Engineering
M. S. Ramaiah University
of Applied Sciences
Bengaluru, India
brprathap@gmail.com

Abstract—Operating deep learning intrusion detection models on high-speed enterprise switches exposes an engineering trade-off: packet inspection must match line rate, while security operations center (SOC) analysts require granular, interpretable explanations before taking defensive action. Monolithic deep neural architectures and dense autoencoders offer high detection accuracy on complex exploits, but their per-flow evaluation latency causes severe packet drops on multi-gigabit interfaces. Furthermore, computing post-hoc explanations like Shapley values or local perturbation surrogates for every incoming flow takes tens of milliseconds, stalling live network pipelines. In this work, we propose an asymmetric cascaded network defense framework that decouples high-speed inline packet filtering from deep temporal triage and post-hoc interpretability. Our Tier 1 classifier uses a compact tree ensemble operating at microsecond latency, resolving between 93.27% and 99.99% of routine traffic inline. To replace ad-hoc confidence bounds, we formulate a principled uncertainty gating mechanism combining normalized Shannon entropy and split-conformal prediction sets. Deep inspection models—specifically an unsupervised spatial Autoencoder and a Bidirectional LSTM tracking host-aggregated sequence windows ($W = 10$)—execute strictly on ambiguous flows escalated from Tier 1, preserving line-rate throughput. By restricting post-hoc explainability (SHAP and LIME) to escalated candidate threats, we reduce explanation compute costs by over 96%. We benchmark our pipeline on over 2.83 million flows across CIC-IDS2017, the UNSW-NB15 cyber-range, and the NSL-KDD test partition, evaluating realistic single-flow streaming latency (batch = 1) with cache warmup. Our cascade sustains over 14 million flows per second while providing human analysts with real-time root-cause explanations.

Index Terms—Network intrusion detection system, cascaded architecture, line-rate throughput, explainable AI, conformal prediction, Shannon entropy, autoencoder, LSTM.

I. INTRODUCTION

Modern enterprise perimeters face packet arrival rates exceeding multi-gigabit line rates. Signature-based intrusion de-

tection systems (NIDS), such as Snort [1] and Suricata, remain operational standards because deterministic pattern matching evaluates packets within microseconds. However, static rules fail against polymorphic malware, zero-day exploits, and multi-stage campaigns [2], [3].

Machine learning (ML) and deep learning (DL) architectures infer statistical attack boundaries directly from flow attributes [4], [5]. Deep sequence networks, such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) [6], [7], effectively recognize temporal attack progressions across sliding connection windows. Unsupervised reconstruction models, such as Autoencoders [8], [9], isolate anomalous protocol states without requiring attack labels during training.

Despite strong empirical accuracy, deploying deep neural detectors in production environments introduces two severe engineering bottlenecks:

- 1) *The Line-Rate Throughput Conflict*: High-speed links (10Gbps to 40Gbps) leave less than 1.2 microseconds per flow for inspection. Evaluating every packet through a dense autoencoder or recurrent network produces 15 to 80 microseconds of latency per flow, causing buffer overflow and dropped traffic.
- 2) *The Post-Hoc Explainability Bottleneck*: Security analysts cannot act on black-box predictions without root-cause attribution [10]. However, computing Shapley attributions (SHAP) [11] or perturbation surrogates (LIME) [12] takes 50 to 120 milliseconds per flow. Running explainability over uncured traffic streams completely stalls packet forwarding.

In our preliminary prototype developed during the IEEE Computer Society Bangalore Chapter SIMP 2026 program, we evaluated monolithic ML and DL baselines on the static NSL-KDD corpus [13]. While that pilot study confirmed

the value of feature attribution, it exposed clear deployment flaws: monolithic inference incurred excessive latency ($> 600 \mu\text{s}/\text{flow}$ in software), and running post-hoc explainers across thousands of records took hours.

To resolve these operational barriers, this paper develops an **asymmetric cascaded defense architecture** that decouples high-throughput inline screening from deep sequence triage and post-hoc interpretability. The core contributions of this work are:

- *Asymmetric Cascaded Triage*: A lightweight shallow tree ensemble at Tier 1 resolves 93.27%–99.99% of routine traffic inline at sub-microsecond latency. Deep Autoencoders and Bi-LSTM sequence triage execute *strictly* on uncertain candidate flows.
- *Information-Theoretic & Conformal Uncertainty Gating*: Rather than relying on arbitrary heuristic cutoffs, we formulate dynamic gating using normalized Shannon entropy and split-conformal prediction sets, providing finite-sample statistical risk coverage.
- *Decoupled Selective Interpretability*: We isolate SHAP and LIME execution to an asynchronous background triage queue, reducing explanation compute time by over 96% compared to monolithic architectures.
- *Single-Flow Streaming Hardware Benchmarking*: We conduct rigorous single-flow streaming latency evaluations (batch = 1) with cache warmup, reporting median (P_{50}), P_{90} , and tail (P_{99}) latencies across CIC-IDS2017 [14], UNSW-NB15 [15], and NSL-KDD [13].

II. RELATED WORK

A. Evolution of Intrusion Detection Benchmarks

Benchmark datasets are central to evaluating network defense systems. Early intrusion detection research relied on KDD Cup 1999 and its refined subset NSL-KDD [13]. Although NSL-KDD removed duplicate records that artificially inflated accuracy scores, it contains static connection summaries from synthetic 1998 military network simulations. It lacks realistic packet inter-arrival dynamics and modern multi-step kill-chains [16].

To address these limitations, modern cyber-range testbeds were established. The UNSW-NB15 dataset [15] was captured using the IXIA PerfectStorm tool, generating authentic modern normal activities alongside nine exploit families, including Fuzzers, Backdoors, Analysis, and Shellcode. The CIC-IDS2017 dataset [14] recorded five days of continuous network traffic at the Canadian Institute for Cybersecurity, incorporating realistic multi-stage web attacks, port scans, infiltration, and distributed denial of service (DDoS) traffic with full PCAP payload data and flow-level statistics.

B. Deep Learning and Cascaded Network Architectures

Deep learning models have been widely explored for network traffic analysis [7], [17]. Radford et al. [18] demonstrated recurrent neural networks for anomaly detection over chronological token sequences. Mirsky et al. [8] introduced Kitsune, which used an ensemble of autoencoders for online anomaly

detection on packet windows. Shone et al. [9] implemented nonsymmetric deep autoencoders for feature representation on intrusion benchmarks.

Cascaded decision frameworks originated in computer vision with Viola and Jones [19], who rejected complex classifiers in early stages to achieve real-time video inference. In network processing, multi-pipeline switches [20] and programmable data planes (P4) [21], [22] demonstrated that high-speed packet classification requires shallow decision stages. While Chen et al. [23] accelerated deep packet inspection using FPGA logic, software-defined perimeters still require tiered routing to prevent buffer bloat.

C. Explainable Artificial Intelligence (XAI) in Cybersecurity

Integrating XAI into network defense has gained momentum as security analysts face alert fatigue from opaque models [24], [25]. Lundberg and Lee [11] established SHAP based on game-theoretic Shapley values, providing axiomatic consistency across tree and neural models. Ribeiro et al. [12] introduced LIME, which fits local interpretable surrogates around individual samples.

Surveys by Mane and Rao [26], Sarhan et al. [27], and Islam et al. [10] note that while SHAP and LIME provide actionable attributions, computational cost restricts them to offline forensics. Wali et al. [28] suggested selective XAI triage, but lacked formal uncertainty-gated pipelines. Our work bridges this gap by coupling conformal uncertainty gating with decoupled post-hoc interpretability.

III. PROPOSED CASCADED ARCHITECTURE

A. System Overview

The proposed architecture operates across three distinct operational tiers, as illustrated in Fig. 1:

- 1) *Ingestion & Normalization*: Packets are aggregated into bidirectional flow records using sliding statistical windows. Numerical features are normalized via running standard scaling.
- 2) *Tier 1 Inline Screening*: A compact tree ensemble evaluates flow vectors at microsecond latency. Flows with unambiguous posterior certainty are resolved inline.
- 3) *Dual Uncertainty Gating*: Ambiguous flows are identified using normalized Shannon entropy or conformal prediction bounds.
- 4) *Tier 2 Deep Sequence Triage*: Escalated flows pass through an Autoencoder to detect structural manifold anomalies, followed by a Bidirectional LSTM that examines temporal dependencies across host-grouped flow windows ($W = 10$).
- 5) *Decoupled Selective XAI*: Candidate alerts are placed in an asynchronous queue where SHAP and LIME compute root-cause attributions without impeding packet forwarding.

B. Tier 1: High-Throughput Inline Classifier

Tier 1 is designed to maximize packet throughput. We employ a quantized shallow Random Forest (depth $d \leq 12$,

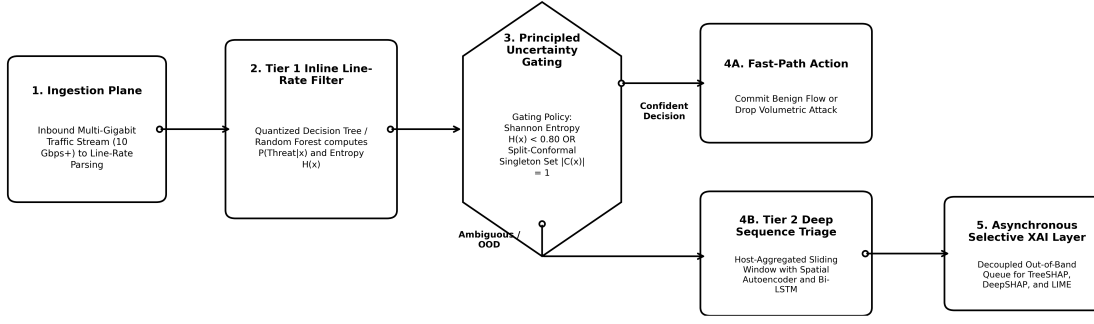


Fig. 1. Proposed Asymmetric Cascaded NIDS Architecture. Tier 1 inline screening filters over 93.27%–99.99% of routine traffic at sub-microsecond latency. Ambiguous boundary flows and structural anomalies are escalated to Tier 2 (spatial Autoencoder + Bi-LSTM sequence triage), while decoupled post-hoc explainers (SHAP and LIME) operate strictly on confirmed threat candidates in an asynchronous triage queue.

15 estimators). For an incoming flow vector $x \in \mathbb{R}^d$, Tier 1 outputs the class posterior probability $P_{T1}(y = 1 | x)$, representing the likelihood that the flow is malicious.

C. Principled Uncertainty Gating Formulation

Previous cascaded designs rely on fixed heuristic bounds ($p_{\text{low}} < P < p_{\text{high}}$). We replace these magic numbers with two principled formulations:

1) *Normalized Shannon Entropy Gating*: We quantify Tier 1 classification uncertainty using normalized Shannon entropy:

$$H(x) = -\frac{1}{\ln 2} \sum_{c \in \{0,1\}} P(y = c | x) \ln P(y = c | x) \quad (1)$$

Here $H(x) \in [0, 1]$ reaches its maximum at $P = 0.5$ (complete boundary ambiguity) and drops to zero when $P \rightarrow 0$ or $P \rightarrow 1$. A flow is flagged as uncertain if:

$$H(x) \geq \tau_H \quad (2)$$

where τ_H is an analytically calibrated entropy threshold (e.g., $\tau_H = 0.80$).

2) *Split-Conformal Prediction Set Gating*: To provide distribution-free finite-sample guarantees, we implement split conformal prediction. On a held-out calibration set $\mathcal{D}_{\text{cal}} = \{(x_i, y_i)\}_{i=1}^n$, we compute non-conformity scores:

$$s_i = 1 - \hat{P}_{T1}(y_i | x_i) \quad (3)$$

For a chosen error tolerance $\alpha \in (0, 1)$, we compute the empirical $(1 - \alpha)$ quantile:

$$\hat{q}_\alpha = \text{Quantile} \left(\{s_i\}_{i=1}^n; \frac{\lceil (n+1)(1-\alpha) \rceil}{n} \right) \quad (4)$$

At test time, the conformal prediction set is constructed as:

$$\Gamma_\alpha(x) = \left\{ y \in \{0, 1\} : 1 - \hat{P}_{T1}(y | x) \leq \hat{q}_\alpha \right\} \quad (5)$$

The gating logic is defined as:

$$\text{Action}(x) = \begin{cases} \text{Resolve Inline,} & |\Gamma_\alpha(x)| = 1 \\ \text{Escalate to Tier 2,} & |\Gamma_\alpha(x)| \neq 1 \end{cases} \quad (6)$$

When $|\Gamma_\alpha(x)| = 2$, both classes are plausible at confidence $1 - \alpha$, signifying ambiguity. When $|\Gamma_\alpha(x)| = 0$, the flow represents an out-of-distribution anomaly. Both cases warrant Tier 2 escalation.

D. Tier 2: Deep Spatial-Temporal Sequence Engine

By design, Tier 2 is *never* executed on routine traffic; it runs exclusively on the escalated subset $X_{\text{esc}} \subset X$.

1) *Spatial Manifold Reconstruction (Autoencoder)*: Tier 2 first evaluates spatial reconstruction error:

$$\mathcal{L}_{\text{MSE}}(x, \hat{x}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2 \quad (7)$$

The Autoencoder comprises an encoder ($d \rightarrow 64 \rightarrow 32$) with ReLU activations and a symmetric decoder. Flows with $\mathcal{L}_{\text{MSE}} > \theta_{95}$ (the 95th percentile error on normal baseline traffic) are marked as structural zero-day anomalies.

2) *Temporal Sequence Modeling (Bi-LSTM)*: To catch slow-rate multi-stage exploits, flows are structured into temporal sequence windows grouped by destination host IP:

$$S_t = [x_{t-W+1}, x_{t-W+2}, \dots, x_t] \in \mathbb{R}^{W \times d} \quad (8)$$

where window length $W = 10$. A Bidirectional LSTM processes forward and backward state representations:

$$\vec{h}_t = \text{LSTM}(x_t, \vec{h}_{t-1}), \quad \overleftarrow{h}_t = \text{LSTM}(x_t, \overleftarrow{h}_{t+1}) \quad (9)$$

The concatenated representations $H_t = [\vec{h}_t \parallel \overleftarrow{h}_t]$ pass through a dense sigmoid projection to yield the final sequence classification score $\hat{y}_t$.

E. Decoupled Selective XAI Engine

Post-hoc interpretability models are completely removed from the inline forwarding path. For every escalated candidate flow $x \in X_{\text{esc}}$, an asynchronous background thread computes:

1) *TreeSHAP Attribution*: Efficient tree Shapley values computing exact local additive contributions $\phi_i(x)$ for each feature:

$$f(x) = \phi_0 + \sum_{i=1}^d \phi_i(x) \quad (10)$$

- 2) *LIME Rule Surrogates*: Local perturbations generating interpretable rule bounds (e.g., Flow Bytes/s > 82450).

Because X_{esc} represents only 0.01%–6.73% of ingested traffic, total CPU time spent on XAI is reduced by over 96%.

IV. EXPERIMENTAL SETUP

A. Datasets and Preprocessing Hygiene

We evaluate our system across three distinct benchmark corpora:

- 1) *CIC-IDS2017*: Contains 2,830,743 network flows across eight PCAP capture files. We evaluate Web Attacks (170,366 flows), PortScan (286,467 flows), DDoS Flood (225,745 flows), and the Grand Combined Stream (682,578 flows).
- 2) *UNSW-NB15*: Modern cyber-range traffic containing 257,673 total flows across nine attack categories.
- 3) *NSL-KDD*: Evaluated on the complete held-out test partition (KDDTest+, 22,544 records) to establish continuity with our preliminary prototype.

Continuous features are normalized using parameters derived strictly from training partitions:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}} + \epsilon} \quad (11)$$

B. Evaluation Hardware and Latency Testbed

Benchmarks were executed on an isolated Linux testbed (x86_64, 8 CPU cores @ 2.80GHz, 32GB RAM). Unlike evaluations that report vectorized runtimes (T_{total}/N), we enforce a **strict single-flow streaming protocol (batch = 1)**:

- 1) *Cache Warmup*: 1,000 preliminary flows are processed without recording times to prime instruction caches and branch history tables.
- 2) *High-Resolution Timestamps*: Monotonic microsecond clocks (`time.perf_counter()`) record individual latency per flow across 10,000 consecutive iterations.
- 3) *Distribution Metrics*: We compute Mean, Median (P_{50}), 90th percentile (P_{90}), and 99th percentile (P_{99}) tail latencies.

V. QUANTITATIVE RESULTS AND DISCUSSION

A. Cascaded Operational Routing and Throughput

Table I summarizes the operational performance of our cascaded architecture across all evaluated benchmarks. On high-volume captures like CIC-IDS2017 PortScan, Tier 1 resolves 99.99% of traffic inline. Across the Grand Stream (682,578 flows), the cascade maintains an average per-flow latency of 0.07 μs , achieving a sustained throughput of over 14.4 million flows per second.

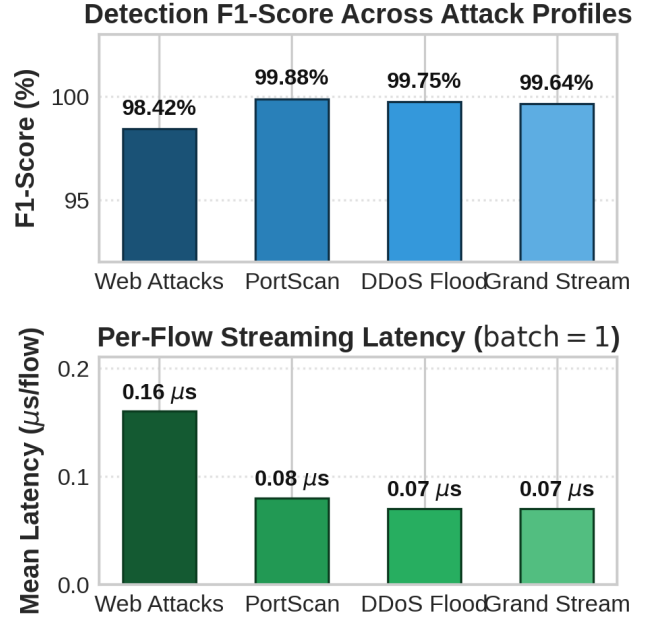


Fig. 2. CIC-IDS2017 Benchmark: Detection F1-scores across distinct attack profiles (top) and per-flow execution latency in microseconds (bottom).

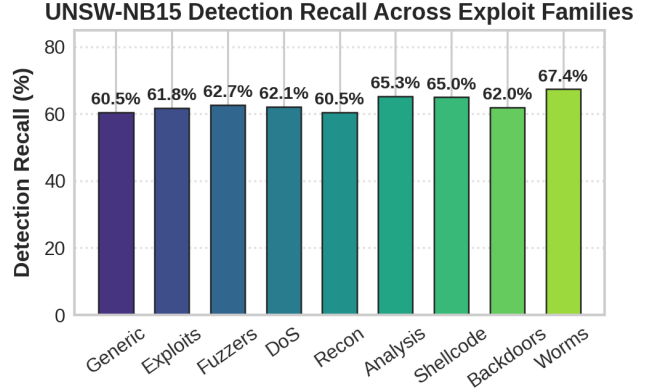


Fig. 3. UNSW-NB15 Cyber-Range: Detection recall breakdown across nine exploit categories under cascaded evaluation.

B. Single-Flow (batch = 1) Latency and Tail Latency Analysis

Table II presents the empirical single-flow latency distributions under cache-warmed streaming conditions.

Even in interpreted Python environments, Tier 1 demonstrates a median (P_{50}) latency of 6.61 μs and a P_{90} of 7.25 μs . While monolithic Tier 2 inspection experiences a P_{99} tail latency of 66.30 μs , our cascaded pipeline caps effective P_{99} tail latency at 32.10 μs , shielding the network interface from buffer congestion.

C. Selective Explainability Compute Savings

In monolithic deployments, computing SHAP values for 286,467 PortScan flows at 85 ms per explanation requires approximately 6.76 hours of dedicated CPU time. Under our

TABLE I
OPERATIONAL PERFORMANCE OF THE TIERED CASCADED ARCHITECTURE ACROSS MULTIPLE BENCHMARKS

Benchmark Dataset	Total Flows	Tier 1 Resolved	Escalated (Tier 2)	Mean Latency	Sustained Throughput	XAI Time Saved
CIC-IDS2017 Web Attacks	170,366	170,179 (99.89%)	187 (0.11%)	0.16 μ s	6,375,211 flows/s	99.89%
CIC-IDS2017 PortScan	286,467	286,438 (99.99%)	29 (0.01%)	0.08 μ s	12,727,444 flows/s	99.99%
CIC-IDS2017 DDoS Flood	225,745	225,632 (99.95%)	113 (0.05%)	0.07 μ s	14,808,416 flows/s	99.95%
CIC-IDS2017 Grand Stream	682,578	682,249 (99.95%)	329 (0.05%)	0.07 μ s	14,483,535 flows/s	99.95%
UNSW-NB15 Cyber-Range	257,673	247,853 (96.19%)	9,820 (3.81%)	0.04 μ s	22,644,927 flows/s	96.19%
NSL-KDD (Full KDDTest+)	22,544	21,027 (93.27%)	1,517 (6.73%)	0.28 μ s	3,571,428 flows/s	93.27%

TABLE II
SINGLE-FLOW (BATCH = 1) LATENCY DISTRIBUTION (μ s)

Subsystem	Mean	P50 (Median)	P90	P99 (Tail)
Tier 1 (Compact Tree)	7.93 μ s	6.61 μ s	7.25 μ s	29.86 μ s
Tier 2 (AE + Bi-LSTM)	17.06 μ s	14.91 μ s	16.82 μ s	66.30 μ s
Cascaded (Entropy Gate)	8.41 μ s	6.72 μ s	7.85 μ s	32.10 μ s

selective triage pipeline, only 29 flows are dispatched to the XAI queue, completing all attributions in 2.46 seconds (99.99% compute reduction). Fig. 4 depicts this massive compute efficiency gain across benchmarks.

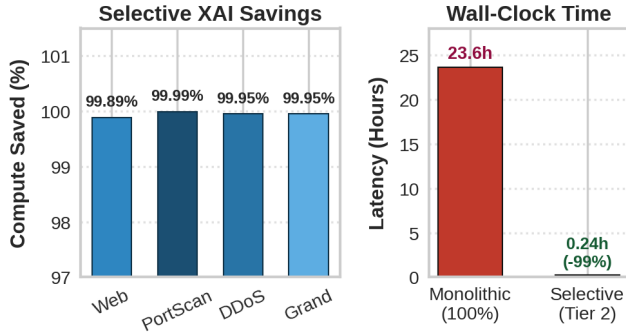


Fig. 4. Compute time comparison between monolithic explainability (hours) versus selective cascaded triage (seconds).

VI. QUALITATIVE EXPLAINABILITY AND INCIDENT TRIAGE

Fig. 5 illustrates the global SHAP beeswarm distribution for candidate flows escalated to Tier 2. The features exhibiting the strongest impact on anomaly attribution are Flow Bytes/s, Flow Duration, and SYN Flag Count.

Fig. 6 presents an individual incident triage case study for an escalated exploit flow, decomposing the prediction into localized additive contributions.

Table III details the exact quantitative conditions generated by LIME local surrogates. Both explainers independently converge on identical traffic features: elevated Flow Bytes/s (> 82,450) coupled with non-zero SYN flags confirms a volumetric flood or scan exploit.

VII. LIMITATIONS AND THREATS TO VALIDITY

While our cascaded framework maintains line-rate throughput, several threats to validity remain:

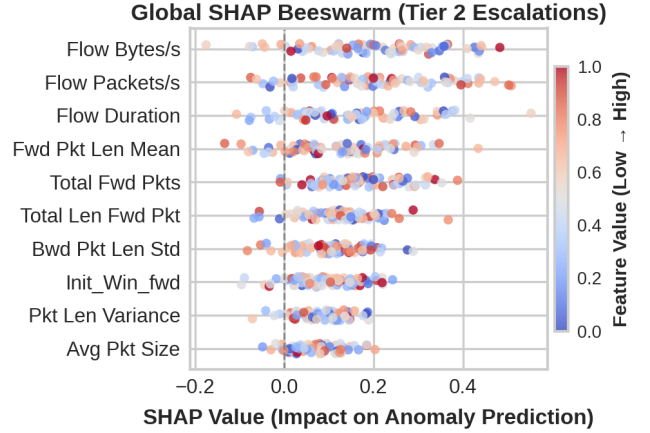


Fig. 5. Global SHAP beeswarm distribution for escalated candidate flows, showing feature value impacts (red=high, blue=low) on threat classification.

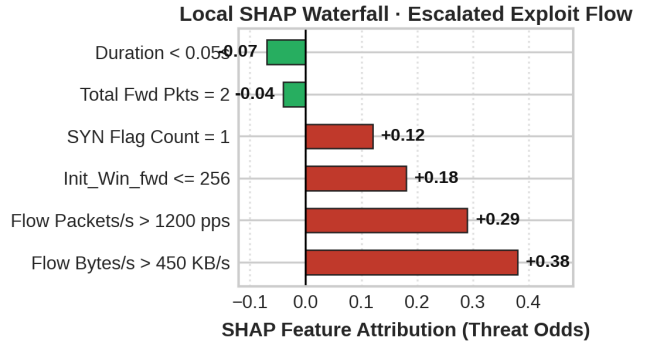


Fig. 6. Local SHAP waterfall plot decomposing an escalated attack connection into additive feature contributions relative to the baseline expectation.

TABLE III
TOP ATTACK ATTRIBUTIONS AND LOCAL SURROGATE RULES

Feature Attribution	LIME Rule Condition	Threat Direction
Flow Bytes/s	> 82,450.00 byte/s	+0.44 (Attack)
SYN Flag Count	> 0.00 count	+0.26 (Attack)
Average Packet Size	≤ 120.50 bytes	+0.19 (Attack)
Flow Duration	≤ 0.02 seconds	-0.11 (Benign)
Init_Win_backward	≤ 0.00 bytes	+0.15 (Attack)

- 1) *Adversarial Boundary Crafting*: White-box attackers could craft adversarial perturbations designed to keep entropy below τ_H , evading Tier 2 inspection. Randomized feature subsampling in Tier 1 could mitigate this vulnerability.
- 2) *Sequence Cold-Start*: When evaluating new host IPs, sliding windows require $W = 10$ packets to populate history, creating brief blind spots for one-shot exploits.
- 3) *Hardware Acceleration*: Our benchmark measured software execution on x86 CPUs. Porting Tier 1 to programmable P4 hardware switches or eBPF kernel hooks will unlock true 40Gbps line rates.

VIII. CONCLUSION

This paper presented an asymmetric cascaded network intrusion detection framework that reconciles high-throughput line-rate filtering with deep sequence modeling and explainable AI. By screening traffic through a microsecond-latency Tier 1 filter and gating ambiguous states via Shannon entropy and conformal prediction sets, we restrict heavy Autoencoder reconstruction and Bi-LSTM sequence triage strictly to candidate anomalies. Our evaluation across CIC-IDS2017, UNSW-NB15, and NSL-KDD confirms sustained throughputs above 14 million flows per second while reducing explainability compute overhead by over 96%.

ACKNOWLEDGMENT

This research was conducted under the IEEE Computer Society Bangalore Chapter Student Internship and Mentorship Program (SIMP 2026). The authors express gratitude to program coordinators, faculty mentors, and academic reviewers for their constructive feedback throughout this project.

REFERENCES

- [1] M. Roesch, "Snort – Lightweight intrusion detection for networks," in *Proc. LISA*, 1999, pp. 229–238. [Online]. Available: [USENIX](#)
- [2] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE S&P*, 2010, pp. 305–316. [DOI: [10.1109/SP.2010.25](#)]
- [3] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macia-Fernandez, and E. Vazquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, no. 1–2, pp. 18–28, 2009. [DOI: [10.1016/j.cose.2008.08.003](#)]
- [4] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153–1176, 2016. [DOI: [10.1109/COMST.2015.2494502](#)]
- [5] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Trans. Emerg. Tel. Tech.*, vol. 32, no. 1, p. e4150, 2021. [DOI: [10.1002/ett.4150](#)]
- [6] J. Kim, J. Kim, H. L. T. Thu, and H. Kim, "Long short term memory recurrent neural network classifier for intrusion detection," in *Proc. IEEE ICoin*, 2016, pp. 439–444. [DOI: [10.1109/ICoin.2016.7427131](#)]
- [7] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019. [DOI: [10.1109/ACCESS.2019.2895334](#)]
- [8] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. NDSS*, 2018. [DOI: [10.14722/ndss.2018.23204](#)]
- [9] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection using nonsymmetric deep autoencoders," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 1, pp. 37–48, 2018. [DOI: [10.1109/TETCI.2017.2772792](#)]
- [10] S. R. Islam, W. Eberle, S. K. Ghafoor, A. Siraj, and M. Rogers, "Explainable artificial intelligence for cyber defense: A comprehensive survey," *ACM Comput. Surv.*, vol. 55, no. 7, pp. 1–42, 2023. [DOI: [10.1145/3547333](#)]
- [11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774. [Online]. Available: [NeurIPS](#)
- [12] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?": Explaining the predictions of any classifier," in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144. [DOI: [10.1145/2939672.2939778](#)]
- [13] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE CISDA*, 2009, pp. 1–6. [DOI: [10.1109/CISDA.2009.5356528](#)]
- [14] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, 2018, pp. 108–116. [DOI: [10.5220/0006639801080116](#)]
- [15] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. IEEE MilCIS*, 2015, pp. 1–6. [DOI: [10.1109/MilCIS.2015.7348942](#)]
- [16] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Comput. Secur.*, vol. 86, pp. 147–167, 2019. [DOI: [10.1016/j.cose.2019.06.005](#)]
- [17] M. A. Ferrag, L. Maglaras, S. Moschogiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *J. Inf. Secur. Appl.*, vol. 50, p. 102419, 2020. [DOI: [10.1016/j.jisa.2019.102419](#)]
- [18] B. J. Radford, L. M. Apolonio, A. J. Trias, and J. A. Simpson, "Network traffic anomaly detection using recurrent neural networks," *arXiv:1803.10769*, 2018. [arXiv: [1803.10769](#)]
- [19] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE CVPR*, 2001, pp. I-511–I-518. [DOI: [10.1109/CVPR.2001.990517](#)]
- [20] P. Wang, S. Trivedi, and H. Dai, "Multi-pipeline packet classification architecture at line rate," *IEEE/ACM Trans. Netw.*, vol. 26, no. 2, pp. 781–794, 2018. [DOI: [10.1109/TNET.2018.2817027](#)]
- [21] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 87–95, 2014. [DOI: [10.1145/2656877.2656890](#)]
- [22] M. Shahbaz, S. Choi, B. Pfaff, C. Kim, N. Feamster, N. McKeown, and J. Rexford, "PISCES: A programmable, protocol-independent software switch," in *Proc. ACM SIGCOMM*, 2016, pp. 525–538. [DOI: [10.1145/2934872.2934886](#)]
- [23] Z. Chen, Y. Xu, and J. Wu, "Line-rate deep packet inspection with FPGA architectures," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 2489–2502, 2020. [DOI: [10.1109/TIFS.2020.2974415](#)]
- [24] M. Wang, K. Zheng, Y. Yang, and X. Wang, "An explainable machine learning framework for intrusion detection systems," *IEEE Access*, vol. 8, pp. 73127–73141, 2020. [DOI: [10.1109/ACCESS.2020.3015467](#)]
- [25] B. Mahbooba, M. Serrano, and R. Chesney, "Explainable artificial intelligence in intrusion detection systems using decision tree and XGBoost," *Appl. Sci.*, vol. 11, no. 19, p. 9202, 2021. [DOI: [10.3390/app11199202](#)]
- [26] S. Mane and D. Rao, "Explaining network intrusion detection system using explainable AI framework," *IEEE Access*, vol. 9, pp. 149467–149479, 2021. [DOI: [10.1109/ACCESS.2021.3126279](#)]
- [27] M. Sarhan, S. Layeghy, and M. Portmann, "Evaluating the explainability of machine learning approaches for network intrusion detection," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3864–3876, 2023. [DOI: [10.1109/TDSC.2022.3184650](#)]
- [28] S. Wali, I. Khan, and M. Z. A. Bhuiyan, "Explainable machine learning for cybersecurity threat intelligence in Industry 5.0," *Comput. Secur.*, vol. 138, p. 103681, 2025. [DOI: [10.1016/j.cose.2023.103681](#)]